Microsoft

AI-200T00A

Develop AI solutions with Azure Database for PostgreSQL

© Copyright Microsoft Corporation. All rights reserved.

Agenda

- Build and query with Azure Database for PostgreSQL
- Implement vector search with Azure Database for PostgreSQL
- Optimize vector search in Azure Database for PostgreSQL

© Copyright Microsoft Corporation. All rights reserved.

Module 1: Build and query with Azure Database for PostgreSQL

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Explain the architecture and key features of Azure Database for PostgreSQL
- Establish secure connections to PostgreSQL using Microsoft Entra authentication and TLS
- Create and manage database schemas including tables, indexes, and constraints
- Write efficient SQL queries for common data operations
- Integrate Azure Database for PostgreSQL into applications using Python

© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Fully managed PostgreSQL on Azure
- Full community edition compatibility
- Handles patching, backups, and high availability automatically
- Compute and storage scale independently
- Built-in PgBouncer connection pooling
- JSONB for flexible schemas alongside relational data
- Extension ecosystem includes pgvector for AI vector search

© Copyright Microsoft Corporation. All rights reserved.

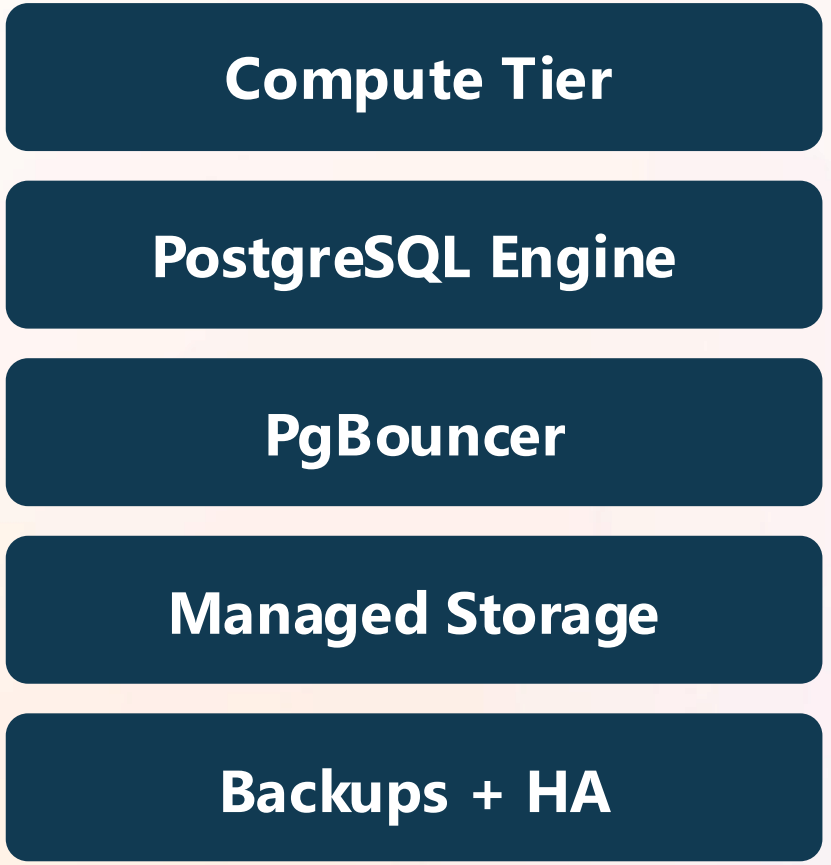
Explore Azure Database for PostgreSQL

Compute tiers:

- Burstable: dev/test, intermittent workloads
- General Purpose: production, steady performance
- Memory Optimized: large in-memory datasets, analytics

Managed capabilities:

- Automated backups with point-in-time restore (up to 35 days)
- Built-in PgBouncer on General Purpose and Memory Optimized tiers
- Extensions: pgvector, pg_trgm, hstore



© Copyright Microsoft Corporation. All rights reserved.

Connect to PostgreSQL

Authentication options:

- Microsoft Entra ID: token-based, no stored passwords
- PostgreSQL native: username/password, store in Key Vault

TLS encryption:

- TLS 1.2+ enforced by default
- Use sslmode=verify-full in production

Network access:

- Public access with firewall rules
- Private access with VNet integration



© Copyright Microsoft Corporation. All rights reserved.

Create and manage schemas

Table design decisions:

- BIGSERIAL for auto-increment, UUID for external IDs
- JSONB for variable-structure metadata
- TIMESTAMPTZ for all temporal data

Constraints enforce data integrity:

- NOT NULL, CHECK, UNIQUE at the database level
- Foreign keys with CASCADE or RESTRICT actions
- Composite primary keys for junction tables

```
CREATE TABLE conversations (
  id BIGSERIAL PRIMARY KEY,
  session_id UUID NOT NULL,
  user_id VARCHAR(255) NOT NULL,
  started_at TIMESTAMPTZ
    DEFAULT CURRENT_TIMESTAMP,
  metadata JSONB DEFAULT '{}'
);

CREATE TABLE messages (
  id BIGSERIAL PRIMARY KEY,
  conversation_id BIGINT NOT NULL
    REFERENCES conversations(id),
  role VARCHAR(50) NOT NULL,
  content TEXT NOT NULL,
  created_at TIMESTAMPTZ
    DEFAULT CURRENT_TIMESTAMP
);
```

© Copyright Microsoft Corporation. All rights reserved.

Query data

PostgreSQL-specific features:

- JSONB operators: `->`, `->>`, `@>`, `?`
- RETURNING clause on INSERT/UPDATE/DELETE
- ON CONFLICT for upsert operations

Performance patterns:

- Keyset pagination over OFFSET for large datasets
- CTEs for readable multi-step queries
- Recursive CTEs for hierarchical data

```
-- Upsert with ON CONFLICT
INSERT INTO user_preferences
  (user_id, key, value)
VALUES ('u1', 'theme', 'dark')
ON CONFLICT (user_id, key)
DO UPDATE SET
  value = EXCLUDED.value,
  updated_at = CURRENT_TIMESTAMP;

-- Keyset pagination
SELECT id, content, created_at
FROM messages
WHERE (created_at, id)
  < (:last_ts, :last_id)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

© Copyright Microsoft Corporation. All rights reserved.

Integrate SDKs and applications

psycopg 3 for Python:

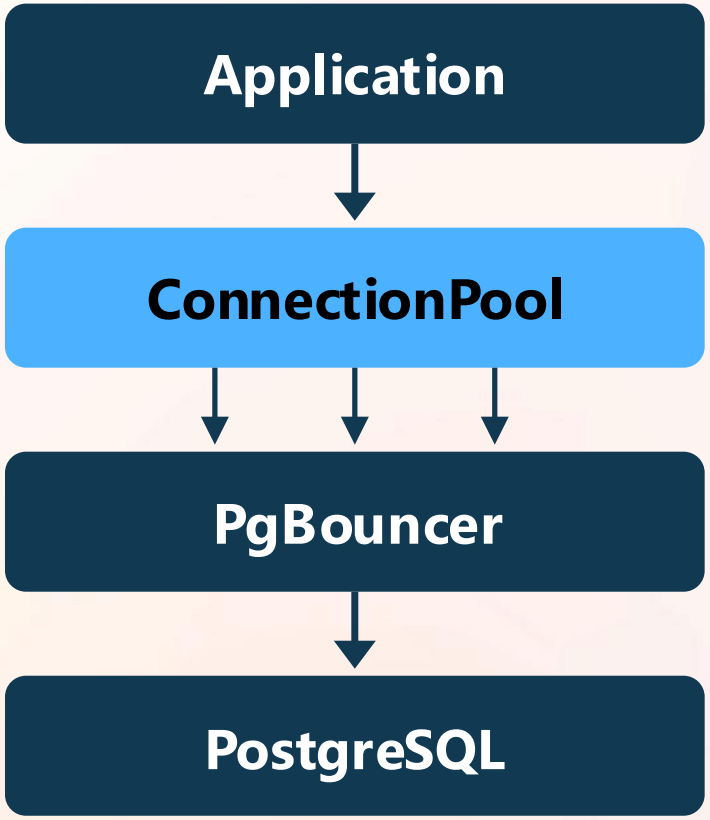
- Parameterized queries with %s placeholders
- Context managers for automatic cleanup

Connection management:

- ConnectionPool for reusable connections
- Timeouts and retry with exponential backoff

Batch performance:

- executemany for hundreds of rows
- COPY command for 10,000+ rows



© Copyright Microsoft Corporation. All rights reserved.

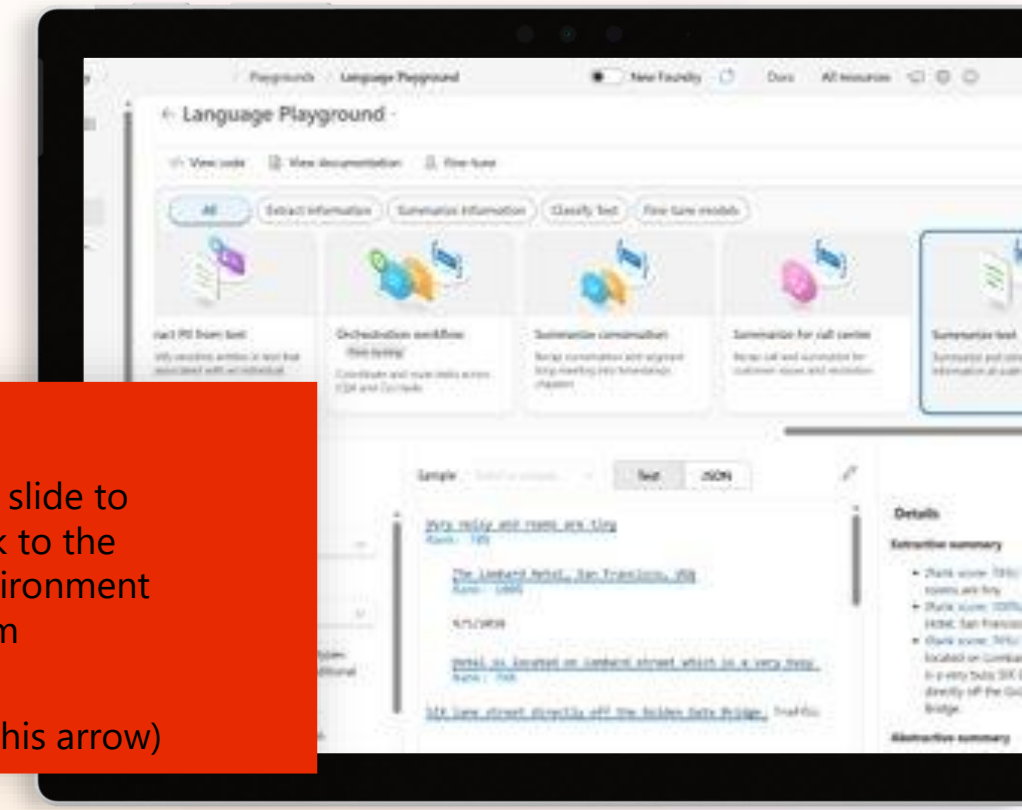
Exercise

Build an agent tool backend on Azure Database for PostgreSQL

In this exercise, you create an Azure Database for PostgreSQL instance that serves as a tool backend for an AI agent. The database stores conversation context and task state that an agent can read and write during operation.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2349114



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 1 knowledge check

1

Your AI app stores conversation metadata with varying structures. Which PostgreSQL data type best supports this?

A. TEXT
B. JSONB
C. VARCHAR(MAX)

2

Your app creates many short-lived database connections during AI inference. What approach improves performance?

A. Open a new connection for each message and close immediately
B. Use a ConnectionPool to maintain reusable connections
C. Keep a single global connection open for the entire app lifetime

© Copyright Microsoft Corporation. All rights reserved.

Module 1 summary

- **Compute tier selection:** Match workload to Burstable (dev), General Purpose (production), or Memory Optimized (analytics)
- **Entra authentication:** Token-based access eliminates stored passwords. Use verify-full SSL in production.
- **Schema design:** BIGSERIAL for auto-increment, JSONB for flexible metadata, constraints enforce integrity at the database level
- **Keyset pagination:** Avoids OFFSET performance penalty on large tables. Track last seen values instead of page numbers.
- **Upsert with ON CONFLICT:** Enables idempotent insert-or-update operations critical for AI state management
- **Connection pooling:** Use psycopg ConnectionPool and built-in PgBouncer to reduce connection overhead for high-throughput workloads

© Copyright Microsoft Corporation. All rights reserved.

Module 2: Implement vector search with Azure Database for PostgreSQL

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Store and query vector embeddings using the pgvector extension
- Execute vector similarity searches using different distance metrics
- Create and manage vector indexes to optimize search performance
- Implement embedding update and refresh strategies
- Build retrieval patterns that integrate PostgreSQL vector search with RAG pipelines

© Copyright Microsoft Corporation. All rights reserved.

Introduction

- pgvector adds vector types and similarity operators directly to PostgreSQL
- Store embeddings alongside relational data without a separate vector database
- Supports semantic search, recommendations, and RAG pipelines
- Two ANN index types (IVFFlat, HNSW) for production-scale performance



© Copyright Microsoft Corporation. All rights reserved.

Store and query embeddings with pgvector

Schema design:

- Define columns with `vector(n)` matching your model's output dimensions
- Store embeddings in the same row as metadata for single-query retrieval

Distance operators:

- `<->` L2 (Euclidean) distance
- `<=>` Cosine distance, most common for text embeddings
- `<#>` Negative inner product

```
CREATE TABLE documents (
    id SERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    content TEXT,
    category TEXT,
    embedding vector(1536)
);
```

© Copyright Microsoft Corporation. All rights reserved.

Perform fast vector similarity search

IVFFlat

- Clusters vectors into partitioned lists
- Requires existing data for creation
- Faster builds, lower memory

```
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

HNSW

- Multi-layer navigable graph structure
- Indexes empty tables incrementally
- Higher recall and query performance

```
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

© Copyright Microsoft Corporation. All rights reserved.

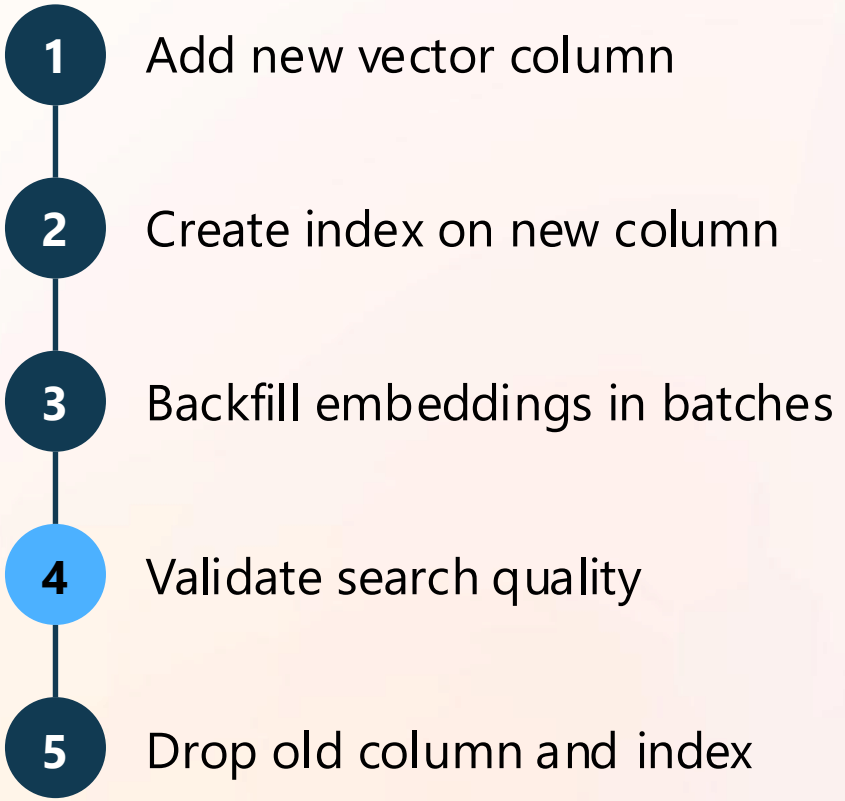
Manage index lifecycle and embedding updates

Index maintenance:

- Monitor with pg_stat_user_indexes for usage and size
- Rebuild when data distribution shifts significantly
- Use CREATE INDEX CONCURRENTLY for zero-downtime rebuilds

Embedding updates:

- Batch updates in 1,000-5,000 row transactions
- Use a stale flag column for async refresh
- Parallel column strategy for model migrations



© Copyright Microsoft Corporation. All rights reserved.

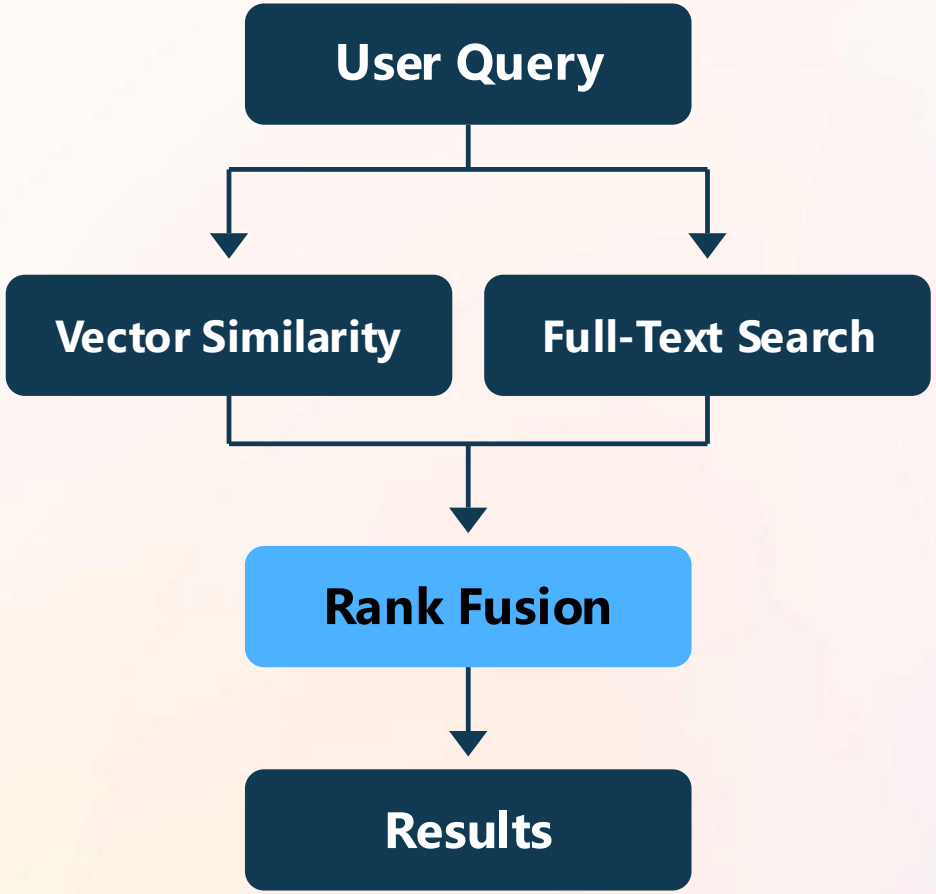
Run vector similarity search for semantic retrieval

Refining results:

- Combine vector similarity with WHERE clauses on metadata
- Set distance thresholds to prevent low-relevance results
- Add B-tree indexes on frequently filtered columns

Hybrid search:

- Combine vector similarity with full-text search for lexical matches
- Reciprocal Rank Fusion balances different scoring scales



© Copyright Microsoft Corporation. All rights reserved.

Implement retrieval patterns for RAG pipelines

RAG retriever design:

- Separate source_documents and document_chunks tables
- Link chunks to sources for citation metadata

Retrieval patterns:

- Fetch adjacent chunks for surrounding context
- Track cumulative token counts to stay within LLM limits
- Measure precision, recall, and MRR to evaluate quality

```
CREATE TABLE document_chunks (
    id SERIAL PRIMARY KEY,
    document_id INT REFERENCES
        source_documents(id),
    chunk_index INT NOT NULL,
    content TEXT NOT NULL,
    embedding vector(1536),
    token_count INT
);
```

© Copyright Microsoft Corporation. All rights reserved.

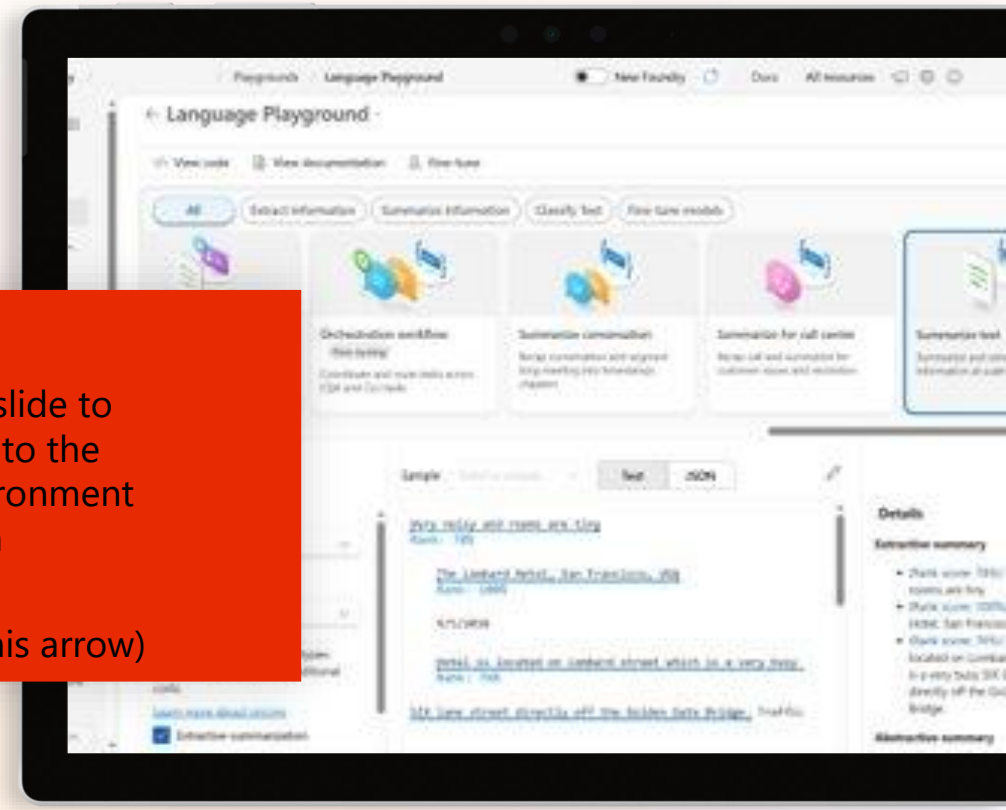
Exercise

Implement vector search on Azure Database for PostgreSQL

In this exercise, you build a product similarity search application using Azure Database for PostgreSQL and the pgvector extension.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2348708



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 2 knowledge check

1 Which pgvector distance operator should you use when your embeddings are normalized to unit length and you want to measure semantic similarity?

A. `<=>` (cosine distance)
B. `<->` (L2 distance)
C. `<#>` (negative inner product)

2 You're building a RAG pipeline that needs to retrieve relevant document chunks quickly from 5 million embeddings. The collection receives occasional batch updates but no real-time inserts. Which index type should you choose?

A. IVFFlat with an appropriate number of lists
B. HNSW with high ef_construction value
C. No index, relying on exact sequential scan

© Copyright Microsoft Corporation. All rights reserved.

Module 2 summary

- **pgvector extension:** adds vector types and distance operators directly to PostgreSQL, no separate vector database needed
- **Distance metric choice:** cosine distance is the default for text embeddings; match operator class to distance operator or indexes won't be used
- **HNSW vs IVFFlat:** HNSW offers better recall and incremental inserts; IVFFlat uses less memory and builds faster for static data
- **Index lifecycle:** rebuild after significant data distribution changes; use concurrent index creation for zero-downtime maintenance
- **Embedding migrations:** parallel column strategy lets you validate new embeddings before switching, with rollback capability
- **RAG retrieval quality:** separate documents from chunks, track token budgets, and measure precision/recall with an evaluation dataset

© Copyright Microsoft Corporation. All rights reserved.

Module 3: Optimize vector search in Azure Database for PostgreSQL

© Copyright Microsoft Corporation. All rights reserved.

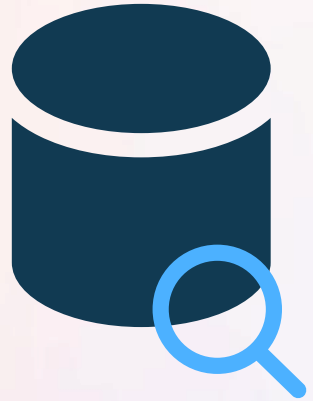
Learning objectives

- Tune PostgreSQL and pgvector configuration parameters to optimize query latency and memory usage for AI workloads
- Select and configure the appropriate vector index type based on dataset size, query patterns, and accuracy requirements
- Design data layouts that optimize vector storage and metadata filtering performance
- Scale Azure Database for PostgreSQL to handle high-volume vector workloads
- Implement connection pooling and session management strategies for AI applications

© Copyright Microsoft Corporation. All rights reserved.

Introduction

- pgvector extension enables vector similarity search in PostgreSQL
- Supports IVFFlat and HNSW approximate nearest neighbor indexes
- Azure Database for PostgreSQL provides managed infrastructure with built-in PgBouncer
- Configuration tuning, indexing, and connection management directly affect query latency
- Optimizations support recommendations, semantic search, and RAG pipelines



© Copyright Microsoft Corporation. All rights reserved.

Tune PostgreSQL for pgvector

Memory and planner tuning:

- `shared_buffers` and `work_mem` sized for vector data volume
- `random_page_cost` lowered to 1.1 for SSD storage
- `effective_io_concurrency` set to 200 for parallel I/O

pgvector search parameters:

- `ivfflat.probes` controls recall vs speed trade-off
- `hnsw.ef_search` controls graph exploration depth
- `EXPLAIN ANALYZE` validates index usage and timing

```
-- Server parameters
SET shared_buffers = '2GB';
SET work_mem = '256MB';
SET random_page_cost = 1.1;
SET effective_io_concurrency = 200;

-- Search parameters (per session)
SET ivfflat.probes = 10;
SET hnsw.ef_search = 100;

-- Validate index usage
EXPLAIN ANALYZE
SELECT * FROM products
ORDER BY embedding <=> $1 LIMIT 10;
```

© Copyright Microsoft Corporation. All rights reserved.

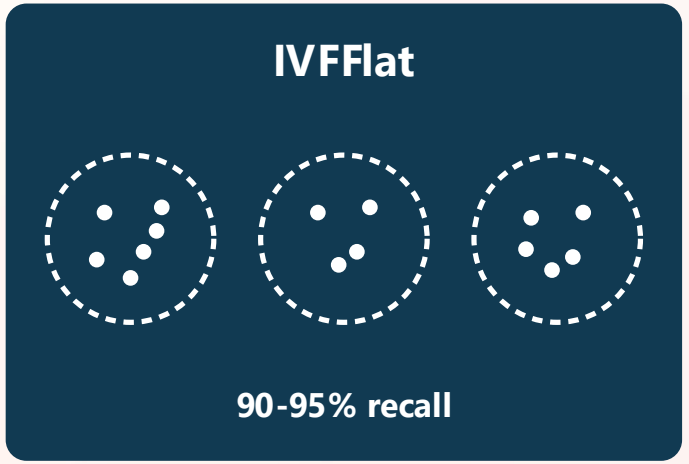
Choose and configure vector indexes

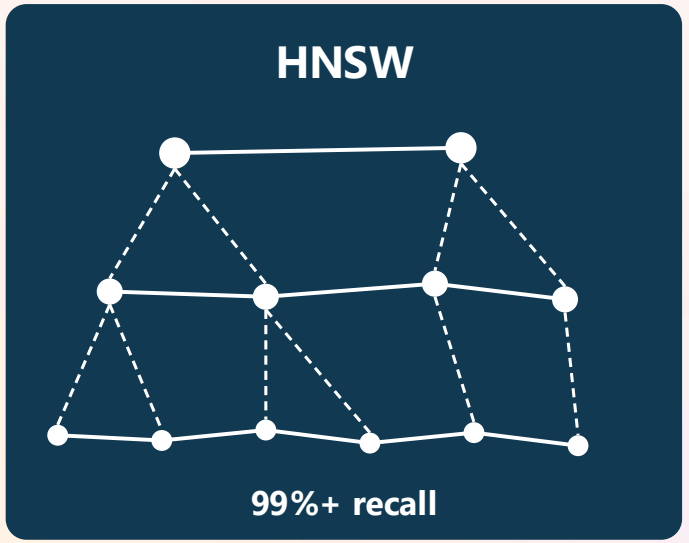
IVFFlat (cluster-based):

- Partitions vectors into lists via k-means clustering
- Fast build times, lower memory usage
- Best for frequent rebuilds or limited memory budgets

HNSW (graph-based):

- Multi-layer navigable graph structure
- Higher recall (99%+), consistent query latency
- Best for read-heavy, accuracy-critical workloads





© Copyright Microsoft Corporation. All rights reserved.

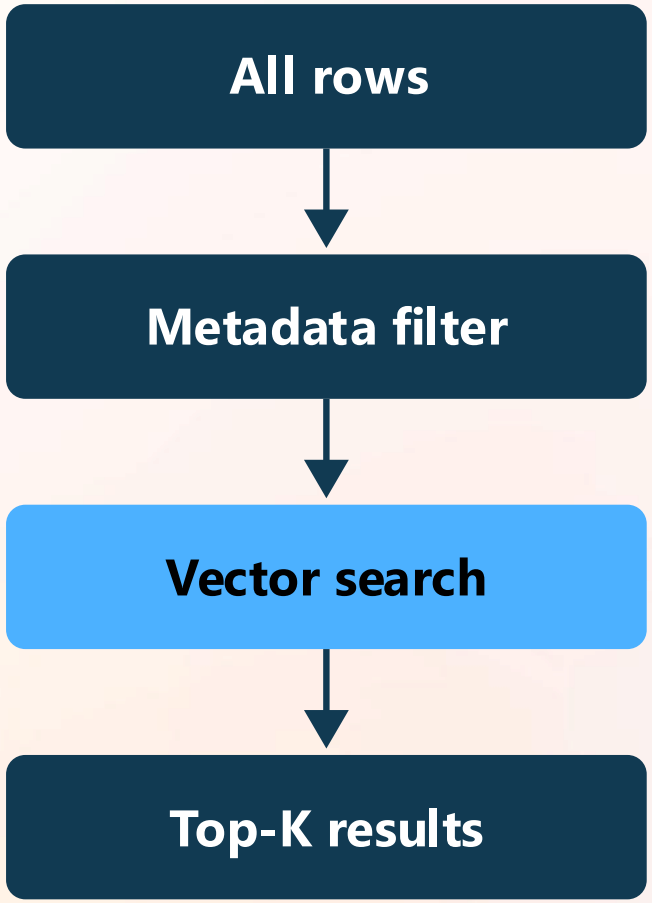
Optimize data layout

Schema design for filtered search:

- Structured columns with B-tree indexes for common filters
- JSONB for dynamic or rarely filtered attributes
- Composite indexes match multi-column filter patterns

Query execution patterns:

- Metadata filters narrow candidates before vector search
- Post-filter pattern for low-selectivity predicates
- Table partitioning for tens of millions of rows



© Copyright Microsoft Corporation. All rights reserved.

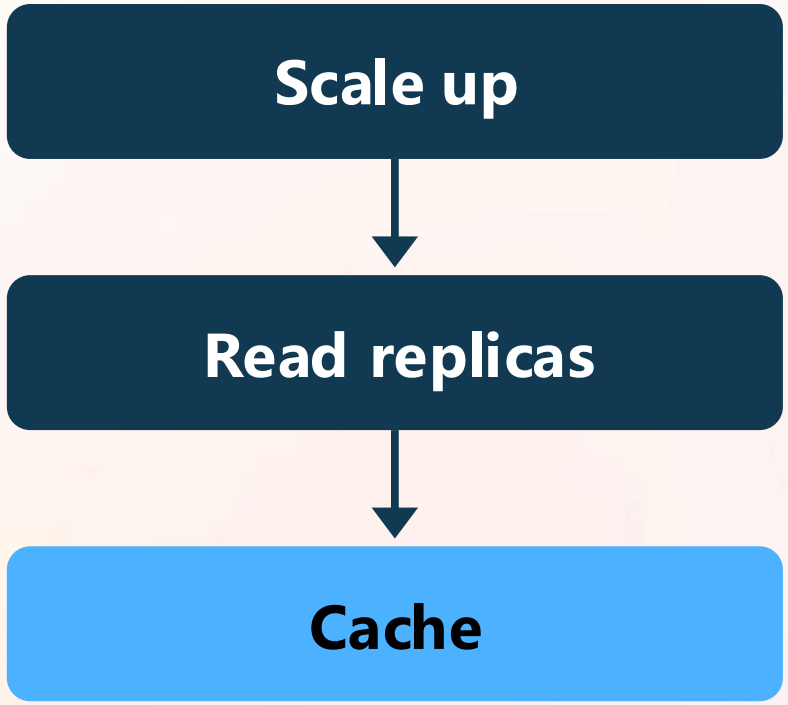
Scale for high-volume workloads

Vertical scaling:

- Memory Optimized tier (8 GB/vCore) keeps indexes cached
- Scale up when single-query latency is too high
- Monitor CPU, memory, and storage I/O thresholds

Horizontal scaling and caching:

- Up to five read replicas for read-heavy distributions
- Azure Cache for Redis for frequently requested data
- Reserved capacity discounts up to 65% for stable baselines



© Copyright Microsoft Corporation. All rights reserved.

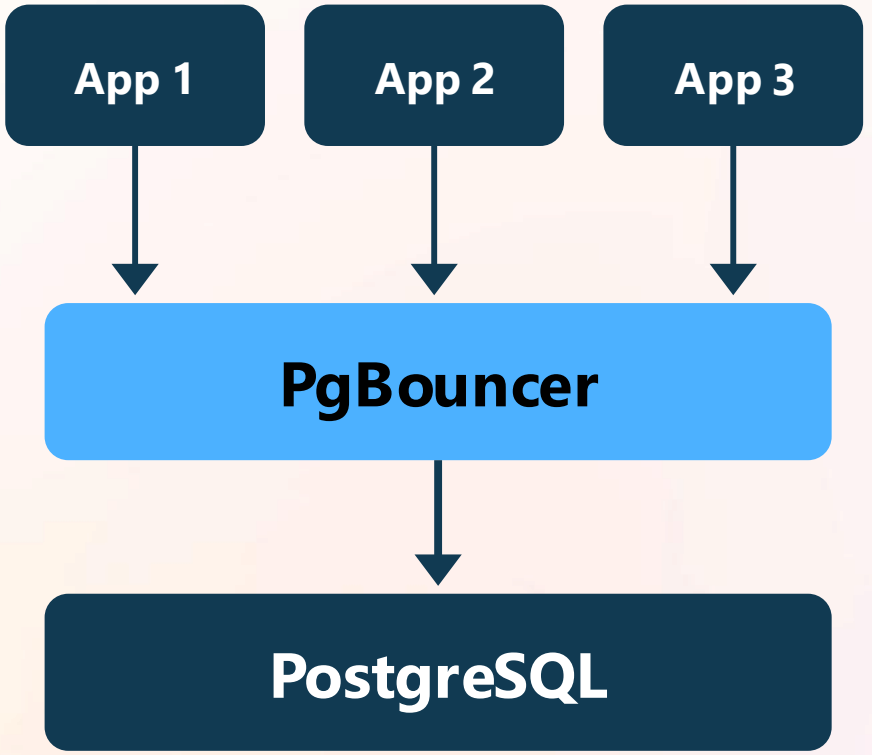
Connection optimization

PgBouncer connection pooling:

- Built-in on General Purpose and Memory Optimized tiers
- Transaction mode multiplexes connections efficiently
- Reduces 50-200ms overhead per new connection

Application-level patterns:

- Pool size balanced across application instances
- SET LOCAL for per-transaction vector search parameters
- Retry with exponential backoff for transient failures



© Copyright Microsoft Corporation. All rights reserved.

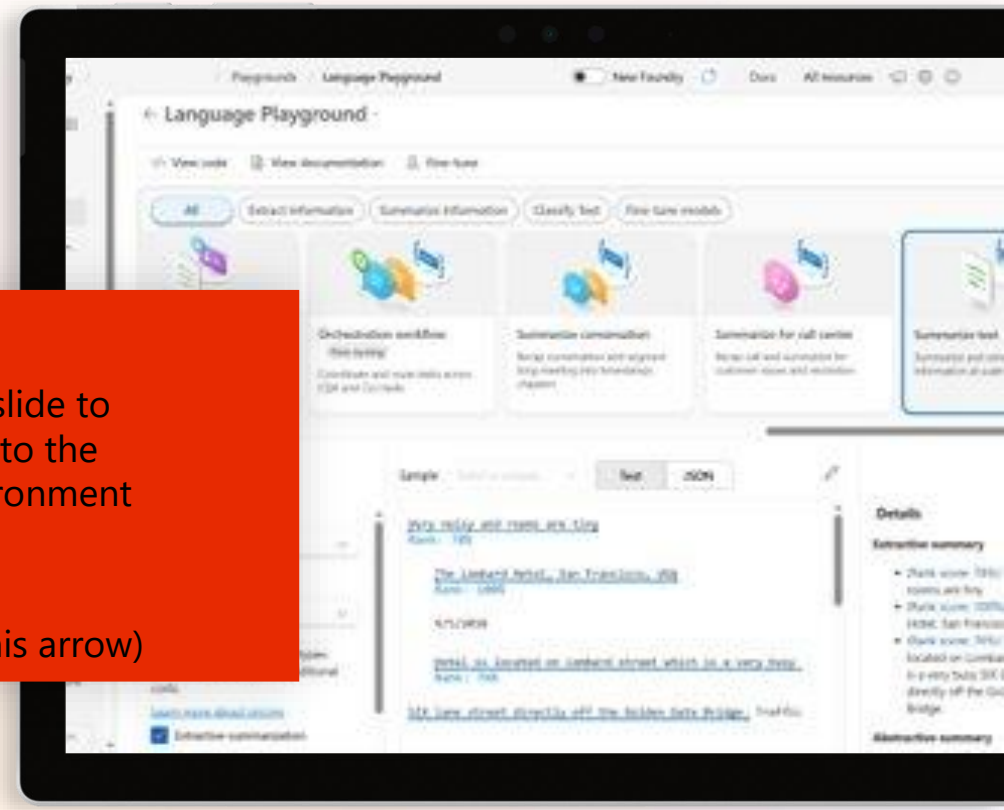
Exercise

Optimize vector search performance in Azure Database for PostgreSQL

In this exercise, you deploy an Azure Database for PostgreSQL instance and optimize it for vector search workloads.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2348919



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 3 knowledge check

1

You need a vector index for 5 million product embeddings with daily full refreshes. Build time must be under 30 minutes. Which index should you choose?

A. IVFFlat with lists set to sqrt(rows)
B. HNSW with m=16 and ef_construction=64
C. HNSW with m=8 and ef_construction=32

2

Your filtered vector search query filters by `category_id` then orders by similarity. The query plan shows a sequential scan. What should you check first?

A. Verify a B-tree index exists on `category_id`
B. Verify the vector index uses the same operator class as the query
C. Increase `hnsw.ef_search` to expand the search space

© Copyright Microsoft Corporation. All rights reserved.

Module 3 summary

- **Memory configuration:** Size `shared_buffers` and `work_mem` to keep vector indexes cached, reducing disk I/O
- **Index selection:** HNSW for high recall and read-heavy workloads, IVFFlat for fast rebuilds and limited memory
- **Metadata filtering:** B-tree indexes on filter columns narrow candidates before distance calculations
- **Scaling strategy**: Vertical scaling first (Memory Optimized tier), then read replicas and caching for volume
- **Connection pooling:** PgBouncer in transaction mode prevents connection exhaustion under high concurrency

© Copyright Microsoft Corporation. All rights reserved.

Group discussion

© Copyright Microsoft Corporation. All rights reserved.

Group discussion questions

- PostgreSQL with pgvector lets you keep vector search alongside relational data in one database. What are the advantages and trade-offs of this approach compared to using a dedicated vector database, and how does that choice affect your team's operations and skillset?
- Embeddings and AI models evolve quickly. Your data store needs to keep up. How do you plan for embedding migrations, index rebuilds, and schema changes without disrupting a production system that serves real-time queries?
- RAG pipelines depend on retrieval accuracy, but "accurate enough" varies by use case. How do you define and measure retrieval quality for your scenario, and at what point do you stop tuning the database and start improving your chunking strategy or prompt engineering instead??

© Copyright Microsoft Corporation. All rights reserved.

End of presentation

© Copyright Microsoft Corporation. All rights reserved.